AIMLCZG546 · SESSION 5 · COMPANION READER

CQRS, RAG, Monolith & Microservices

Four Architectural Patterns That Power Modern ML Systems

Session 5 · Read alongside the lecture slides · Every pattern explained with worked examples

How to use this companion. **Blue** = intuition. **Amber** = everyday analogy. **Green** = worked scenario. **Red** = pitfall. **Purple** = one line to remember.

1 CQRS: Command Query Responsibility Segregation

CQRS is an architectural pattern that separates operations that *change* state (commands) from operations that *read* state (queries). Each side can then be optimised independently.

1.1 The CQS Principle (Origin)

Bertrand Meyer stated the **Command–Query Separation (CQS)** principle in the 1980s while designing the Eiffel language: *"Every method should either be a command that performs an action, OR a query that returns data, but NOT both."* A query must not have side effects. CQRS applies this at the architectural level.

The intuition

A bank ATM illustrates CQS naturally. Pressing "Check Balance" is a query: it returns a number and changes nothing. Pressing "Withdraw $100" is a command: it changes state (deducts money) and returns nothing meaningful. Mixing them — "withdraw $100 and also return the new balance in the same atomic call" — creates subtle consistency problems at scale.

1.2 CQRS in Data Systems

Traditional database architectures use the same data model for reads and writes. Under heavy concurrent load, reads and writes compete for the same locks:

- **Shared Lock (S)** — required for reads; multiple readers can hold simultaneously.
- **Exclusive Lock (X)** — required for writes; no other reader or writer allowed.

CQRS solves this by maintaining *two separate data stores*:

- **Write Store** (command side) — normalised, consistent, optimised for writes (e.g., PostgreSQL).
- **Read Store** (query side) — denormalised, cached, optimised for reads (e.g., Redis, Elasticsearch).

An event stream (Kafka, Kinesis) or message broker (RabbitMQ) propagates changes from the write store to the read store asynchronously.

1.3 Eventual Consistency

The price of CQRS is **eventual consistency**: the read store reflects the write store *eventually*, not immediately.

★ Everyday picture

When GPT-4o is updated at noon, users in Mumbai see the new model immediately. Users in São Paulo may still be using the cached old model for another 3 minutes while the update propagates. Eventually all users get the new model. No two servers are ever guaranteed to be identical at exactly the same millisecond.

1.4 CQRS in ML Systems

In ML architectures, the CQRS split maps naturally:

CQRS Side	ML Operation	Technology
Command (write)	Trigger retraining, update feature store	Airflow, Prefect, Kafka
Command (write)	Log prediction + ground truth	PostgreSQL, BigQuery
Query (read)	Serve predictions via REST API	Redis cache, FastAPI
Query (read)	Dashboard: model metrics, drift reports	Elasticsearch, Grafana

Worked example: CQRS churn prediction system

Query path: Loan officer asks: "What is the predicted churn risk for customer 12345?" Request → API Gateway → Query Service → Redis Cache (cache hit: 0.3ms) → returns `{"customer_id":"12345","churn_risk":0.73,"label":"HIGH"}`.

Command path: Customer 12345 just signed up for the premium tier.
Event published to Kafka: `customer.tier_changed`.
Feature Store Updater consumes event → updates feature: `tier="premium"` in PostgreSQL write store.
Cache Invalidation Service consumes event → deletes Redis key for customer 12345.
Model Retraining Trigger monitors drift → schedules weekly retrain.

Eventual consistency: For the next 30 seconds while Redis is being updated, the query path may still return the old churn risk (0.73). After propagation completes, it returns 0.61 (lower risk because the customer just upgraded). This 30-second window is the consistency gap — acceptable for a risk dashboard, unacceptable for a real-time fraud block.

✓ Key takeaway

CQRS = separate the write path (training, feature updates, logging) from the read path (predictions, dashboards). Each side scales and optimises independently.

2 RAG: Retrieval-Augmented Generation

RAG is an architecture that enhances an LLM by grounding its responses in external, up-to-date, or proprietary data. Instead of relying solely on what was baked into the LLM during training,

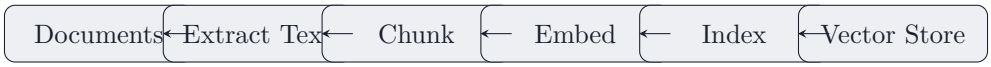
RAG fetches relevant information first and injects it into the prompt.

The intuition

An LLM without RAG is like a student answering an open-book exam from memory only. RAG gives the student the textbook: they look up the relevant pages before writing their answer. The answer is more accurate, more specific, and doesn't require the student to have memorised every fact.

2.1 The Ingestion Pipeline (Pipe-and-Filter)

This is the write side of RAG. It runs once and whenever documents are updated.



2.2 The Query Pipeline (CQRS Query Side)

This runs for every user request, in real time.

Worked example: RAG query for shoe return policy

User query: "What is the return policy for shoes?"

Step 1 — Embed query: Query embedded into 384-dim vector $\mathbf{q} \in \mathbb{R}^{384}$ using `all-MiniLM-L6-v2`.

Step 2 — Semantic search: ChromaDB computes cosine similarity between $\mathbf{q}$ and all stored chunk embeddings using the HNSW index. Top-3 results:
Chunk 47: "Returns accepted within 30 days for footwear..." (similarity: 0.91)
Chunk 12: "Original packaging required for shoe returns..." (similarity: 0.87)
Chunk 83: "Sale items, including shoes on discount, are non-returnable..." (similarity: 0.81)

Step 3 — Build prompt:

```
Context: [Chunk 47 text] [Chunk 12 text] [Chunk 83 text]
Question: What is the return policy for shoes?
Answer based only on the context above.
```

Step 4 — LLM answer: "Shoes can be returned within 30 days in their original packaging. Sale items are non-returnable."

No hallucination: The LLM cannot invent a policy because the instruction says "answer based only on the context."

Vector Stores: ChromaDB (local, open-source, SQLite + HNSW), Pinecone (managed, scalable), FAISS (Facebook, fast in-memory). **Embedding models:** OpenAI Embeddings, SBERT, DPR.

3 Monolithic Architecture

A **monolithic** application packages all its functionality into a single deployable unit. The classic example is the FTGO (Food to Go) app: Order Management, Restaurant Management, Delivery Management, Customer Management, and Payment Processing all live in one WAR file connecting to one MySQL database.

Property	Advantage	Disadvantage
Deployment	Single WAR file; simple CI/CD	Full redeploy for any change
Communication	In-process; fast (no network)	All modules share same failure domain
Scalability	One unit scales together	Cannot scale individual bottleneck components
Technology	One language, one framework	Technology lock-in; new tech = rewrite
Complexity	Easy to understand at first	Grows unmanageable as team/features grow

★ Everyday picture

A monolith is like a Swiss Army knife: one tool, convenient, always in one piece. Great for one person camping. Terrible when 50 people each need a different blade simultaneously and you can only sharpen the whole knife at once.

Monolithic ML Pattern: A research prototype or early-stage ML product often starts as a monolith. The web app, model loading, preprocessing, and prediction logic all live in one Flask application. Issues: updating the model requires full redeployment (risky), single point of failure, memory pressure from loading multiple large models in one process.

4 Microservices Architecture

Microservices decompose an application into small, independent services, each owning its own database and exposing its own API. Each service is deployable independently.

The intuition

A microservices architecture is like a food court. Each stall (pizza, sushi, tacos) is independently owned, staffed, and managed. A long queue at the pizza stall doesn't affect the sushi stall. You can renovate the taco stall without closing the whole court. Each stall uses its own kitchen equipment.

The underlying principle is Robert C. Martin's **Single Responsibility Principle**: *"Gather together things that change for the same reason; separate things that change for different reasons."*

Worked example: Scaling cost: monolith vs microservices

FTGO app handles 1,000 requests/sec normally. During lunch peak, the Order Service gets 5,000 requests/sec, but other services remain at baseline.

Monolith: Must scale the entire application 5×. Cost: 5× compute for ALL services.
Microservices: Scale only the Order Service 5×. Restaurant, Delivery, Payment services stay at 1×.
Cost ratio: If the app has 5 equal services, monolith cost $= 5 \times 5 = 25$ units. Microservices cost $= 5 + 1 + 1 + 1 + 1 = 9$ units. Microservices uses **64 % less compute** during peak.

Worked example: ML microservices demo

A fraud detection system decomposed into three services:

Gateway Service (port 8000): Receives transaction POST requests, routes to downstream services.
Model Service (port 8001): Loads the fraud detection model, returns

`{"fraud_probability": 0.94}`.
Logging Service (port 8002): Receives prediction + transaction, writes to PostgreSQL for drift monitoring.

Each service has its own Docker container. Updating the model (retraining with new fraud patterns) deploys only the Model Service container. The Gateway and Logging services keep running without interruption.

✗ Watch out

Microservices add distributed systems complexity: network latency, service discovery, distributed tracing, and the need for an API gateway. Start with a monolith, identify the bottleneck, and extract *that one service*. Don't build microservices from day one for a two-person team.

✓ Key takeaway

Microservices enable independent scaling, deployment, and technology choice per service. The trade-off is operational complexity that a monolith avoids.